■ Mushroom Business Management & E-Commerce Platform (Mushroom ERP)

Document 04: Prisma ORM Schema Specification & Type Architecture

Project Name: SporeERP
Document Code: DOC-04-PRISMA
Version: 1.0.0-PROD
Database Dialect: MariaDB 11.x (MySQL compatible)

---

1. Overview & ORM Configuration

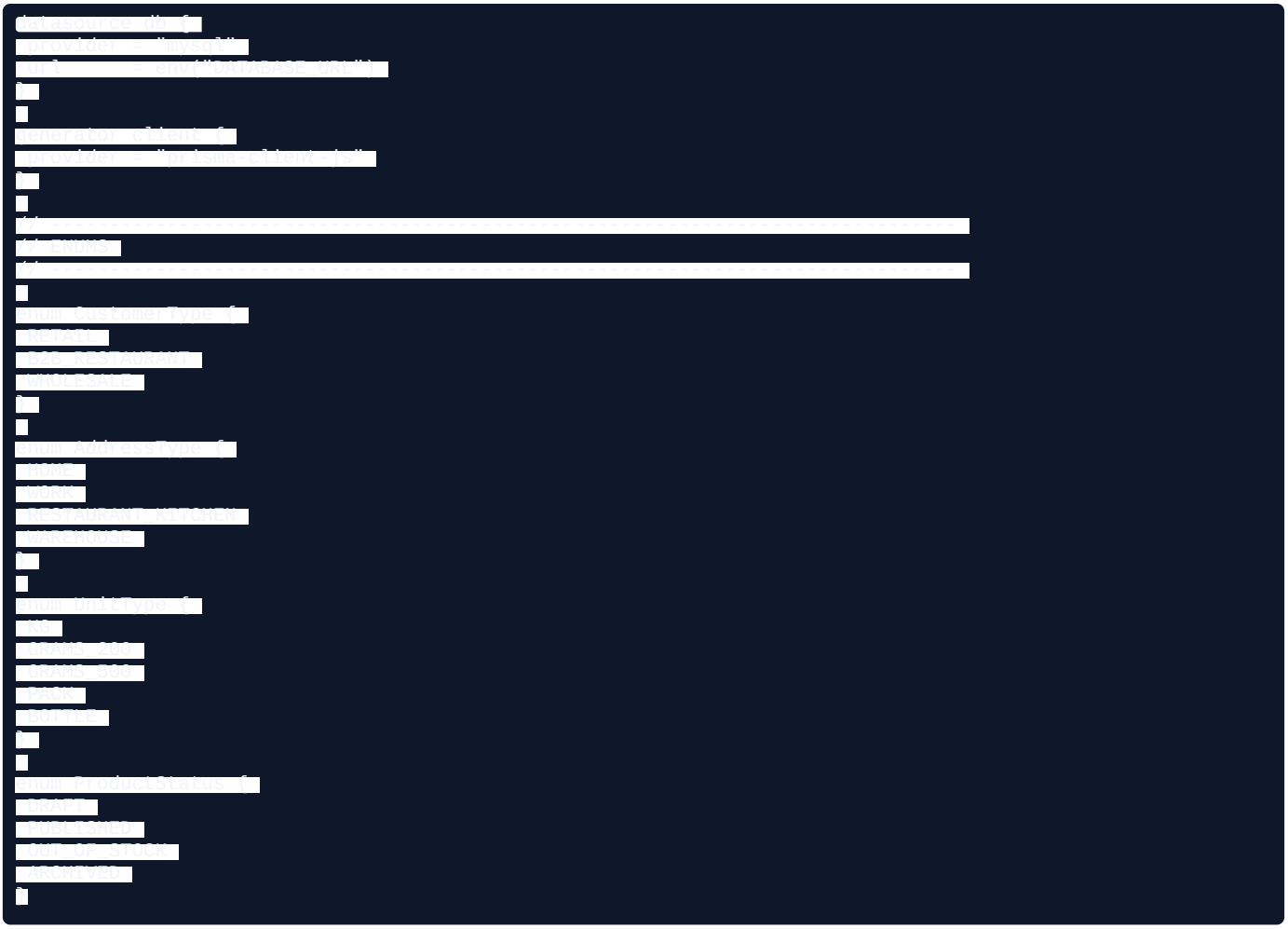
The SporeERP database access layer is driven by Prisma ORM. It establishes strong runtime type checking, automatic TypeScript client generation, and structured database migrations.

Key Highlights:

1. **Explicit Column Mappings:** Using `@map("column_name")` and `@@map("table_name")` ensures MariaDB follows snake_case conventions while TypeScript code uses idiomatic camelCase.
2. **Referential Actions:** Strict enforcement of `onDelete: Cascade` for dependent sub-items (e.g., `OrderItem`, `CustomerAddress`) and `onDelete: Restrict` for critical audit entities (e.g., `Order`, `Product`, `ProductionBatch`).
3. **Optimized Indexes:** Composite and single-column indexes on high-frequency filtering fields (`status`, `customerId`, `createdAt`, `batchId`).

---

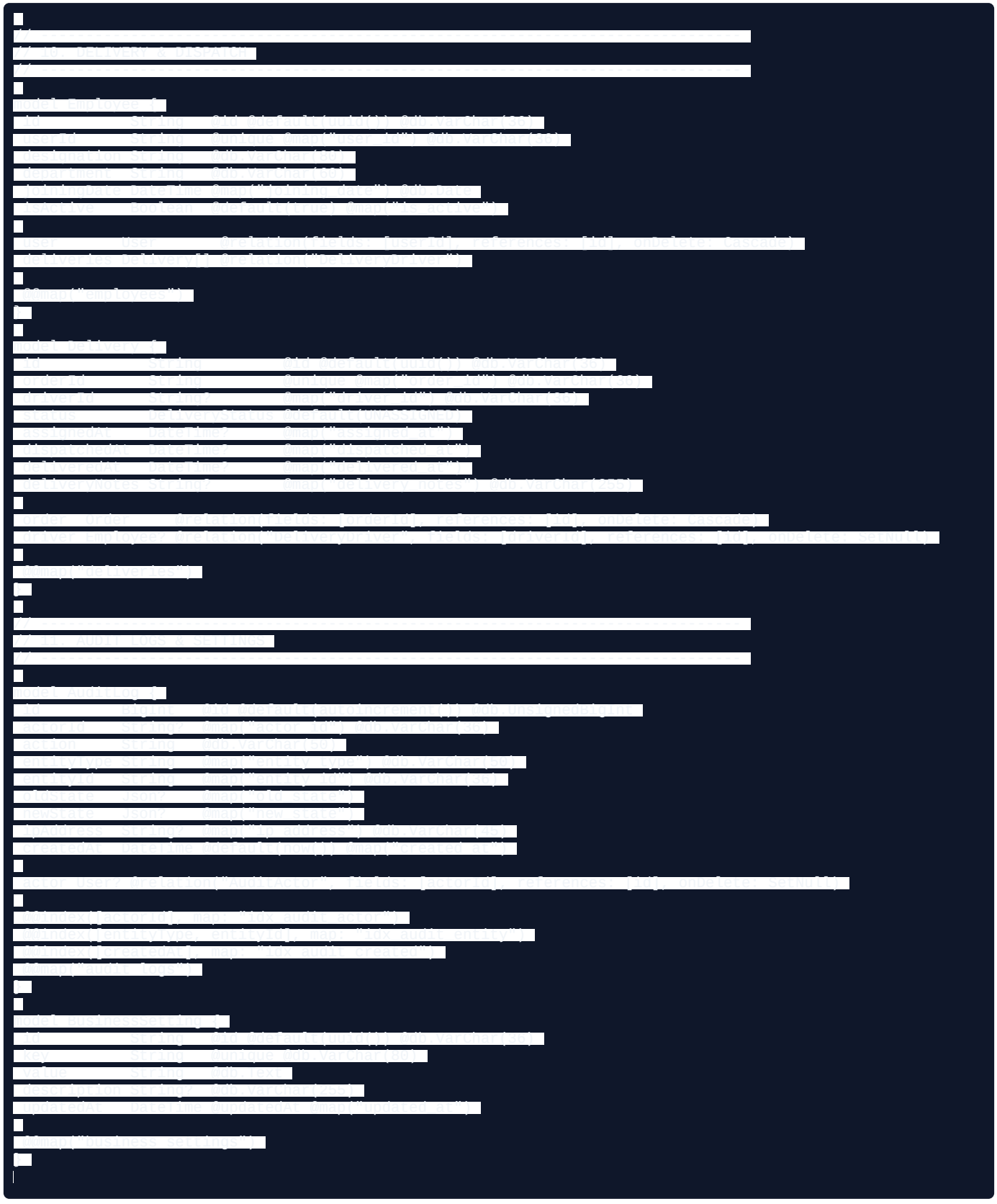
2. Complete Prisma Schema Model Listing



[redacted]

[illegible]

[redacted]

```
@@map("orders")
}

model OrderItem {
  id          String  @id @default(uuid()) @db.VarChar(36)
  orderId     String  @map("order_id") @db.VarChar(36)
  productId   String  @map("product_id") @db.VarChar(36)
  quantity    Decimal @db.Decimal(10, 2)
  unitPrice   Decimal @map("unit_price") @db.Decimal(12, 2)
  totalPrice  Decimal @map("total_price") @db.Decimal(12, 2)

  order   Order   @relation(fields: [orderId], references: [id], onDelete: Cascade)
  product Product @relation(fields: [productId], references: [id], onDelete: Restrict)

  @@map("order_items")
}

model OrderStatusHistory {
  id        String      @id @default(uuid()) @db.VarChar(36)
  orderId   String      @map("order_id") @db.VarChar(36)
  status    OrderStatus
  notes     String?     @db.VarChar(255)
  changedAt DateTime    @default(now()) @map("changed_at")

  order Order @relation(fields: [orderId], references: [id], onDelete: Cascade)

  @@map("order_status_history")
}

// ============================================================
// 5. PAYMENTS & INVOICES
// ============================================================

model Payment {
  id            String        @id @default(uuid()) @db.VarChar(36)
  orderId       String        @map("order_id") @db.VarChar(36)
  transactionId String?       @map("transaction_id") @db.VarChar(100)
  amount        Decimal       @db.Decimal(12, 2)
  method        PaymentMethod @default(UPI) // RAZORPAY
  status        PaymentStatus @default(PENDING)
  gateway       String?       @db.VarChar(50)
  createdAt     DateTime      @default(now()) @map("created_at")

  order Order @relation(fields: [orderId], references: [id], onDelete: Cascade)

  @@map("payments")
}

// ============================================================
// 6. INVENTORY & STOCK TRANSACTIONS
// ============================================================

model InventoryItem {
  id          String   @id @default(uuid()) @db.VarChar(36)
  productId   String   @unique @map("product_id") @db.VarChar(36)
  itemName    String   @map("item_name") @db.VarChar(100)
  itemCategory ItemCategory @default(FINISHED_PRODUCT) @map("item_category")
  availableStock Decimal @default(0) @map("available_stock") @db.Decimal(10, 2)
  reservedStock  Decimal @default(0) @map("reserved_stock") @db.Decimal(10, 2)
  unit        String   @default("KG") @db.VarChar(20)
  reorderThreshold Decimal @default(0) @map("reorder_threshold") @db.Decimal(10, 2)
  updatedAt   DateTime @updatedAt @map("updated_at")

  product Product @relation(fields: [productId], references: [id], onDelete: Restrict)

  @@map("inventory_items")
}

model InventoryTransaction {
  id              String          @id @default(uuid()) @db.VarChar(36)
  inventoryItemId String          @map("inventory_item_id") @db.VarChar(36)
  transactionType InventoryTxType @map("transaction_type")
  quantity        Decimal         @db.Decimal(10, 2)
  balanceAfter    Decimal         @map("balance_after") @db.Decimal(10, 2)
  referenceType   String?         @map("reference_type") @db.VarChar(50)
  referenceId     String?         @map("reference_id") @db.VarChar(36)
  reason          String?         @db.VarChar(255)
  performedBy     String          @map("performed_by") @db.VarChar(36)
  createdAt       DateTime        @default(now()) @map("created_at")
}
```

```
  createdAt     DateTime @default(now()) @map("created_at")
}

model HarvestBatch {
  id            String   @id @default(cuid())
  harvestDate   DateTime @map("harvest_date")
  quantity      Float
  unit          String   @default("kg")
  notes         String?
  recorder      User     @relation("HarvestRecorder", fields: [harvestedBy], references: [id], onDelete: Restrict)
  harvestedBy   String   @map("harvested_by")
  createdAt     DateTime @default(now()) @map("created_at")

  @@index([harvestDate], map: "idx_harvests_date")
  @@map("harvests")
}

// ============================================================
// 5. SUPPLIER & PURCHASE
// ============================================================

model Supplier {
  id                String   @id @default(cuid()) @db.VarChar(36)
  name              String   @db.VarChar(255)
  contactPerson     String?  @map("contact_person") @db.VarChar(100)
  phone             String?  @db.VarChar(20)
  email             String?  @db.VarChar(255)
  materialsSupplied String   @map("materials_supplied") @db.VarChar(255)
  notes             String?  @db.VarChar(255)
  purchases         Purchase[]

  @@map("suppliers")
}

model Purchase {
  id              String         @id @default(cuid()) @db.VarChar(36)
  purchaseNumber  String         @unique @map("purchase_number") @db.VarChar(50)
  supplier        Supplier       @relation(fields: [supplierId], references: [id])
  supplierId      String         @map("supplier_id") @db.VarChar(36)
  purchaseDate    DateTime       @map("purchase_date") @db.Date
  subtotal        Decimal        @db.Decimal(10, 2)
  taxAmount       Decimal        @default(0) @map("tax_amount") @db.Decimal(10, 2)
  totalAmount     Decimal        @map("total_amount") @db.Decimal(10, 2)
  paymentStatus   PaymentStatus  @default(UNPAID) @map("payment_status")

  items           PurchaseItem[]

  @@map("purchases")
}

model PurchaseItem {
  id          String   @id @default(cuid()) @db.VarChar(36)
  purchaseId  String   @map("purchase_id") @db.VarChar(36)
  materialName String  @map("material_name") @db.VarChar(100)
  quantity    Decimal  @db.Decimal(10, 2)
  unit        String   @default("kg") @db.VarChar(20)
  unitPrice   Decimal  @map("unit_price") @db.Decimal(10, 2)
  totalPrice  Decimal  @map("total_price") @db.Decimal(10, 2)

  purchase    Purchase @relation(fields: [purchaseId], references: [id], onDelete: Cascade)

  @@map("purchase_items")
}

// ============================================================
// 6. EXPENSES & AUDIT
// ============================================================

model Expense {
  id            String          @id @default(cuid()) @db.VarChar(36)
  category      ExpenseCategory @default(RAW_MATERIAL)
  amount        Decimal         @db.Decimal(10, 2)
  description   String          @db.VarChar(255)
  expenseDate   DateTime        @map("expense_date") @db.Date
  paymentMethod String          @default("UNPAID") @map("payment_method") @db.VarChar(30)
  receiptUrl    String?         @map("receipt_url") @db.VarChar(255)
  recordedBy    String          @map("recorded_by") @db.VarChar(36)
  createdAt     DateTime        @default(now()) @map("created_at")

  recorder      User            @relation("ExpenseRecorder", fields: [recordedBy], references: [id], onDelete: Restrict)

  @@index([category], map: "idx_expenses_category")
  @@index([expenseDate], map: "idx_expenses_date")
  @@map("expenses")
}
```